

Index

1. Statement of Confidentiality.....	2
2. Engagements Contacts.....	2
3. Executive summary.....	2
4. Approach.....	2
5. Assignement.....	3
6. Scope.....	3
7. Assessment summary.....	3
8. Detailed Walkthrough.....	4
8.1 Reconnaissance.....	4
8.2 Exploitation.....	4
8.3 Post – Exploitation.....	5
8.3.1 Root`s flag.....	6
8.3.2 User`s flag.....	6
9. Remediation summary.....	6
10. Appendix A: Finding severities.....	7
11. Appendix B : Tools.....	8
12. Appendix C : Technical report.....	8

1. Statement of Confidentiality

This report was produced as a part of a personal penetration testing laboratory exercise.

The information contained in this document is intended solely for a educational and professional portfolio purposes. Any IP addresses , credential, flag and system information referenced herein belong to the target laboratory environment and must not be used against unauthorized systems.

Distribution of this report should be limited to educational, recruitment and professional development purposes.

2. Engagments Contacts

Assessor Contact			
Assessor name	Title	Project	Assessor email
X	Project Director	Vaccine	X@htbcompany.htb
José Martín	Penetration Tester	Vaccine	Jose@htbcompany.htb

3. Executive summary

A penetration test was conducted againsts the Vaccine virtual machine from Hack the Box.

The objective of the assessment was to identify vulnerabilities that could allow unauthorized access to the target system and evaluate the impact of a successful compromise.

The test demonstrated weaknesses in web application security, credential managment and operating system hardening.

4. Approach

The assessment followed a standard penetration testing methodology:

1. Reconnaissance
2. Service enumeration
3. Web application enumeration
4. Vulnerability research
5. Initial - Exploitation
6. Post - Exploitation
7. Privilege escalation
8. Evidencie coleccion

Each phase was documented and validated through technical evidence.

5. Assignment

Assignment			
From	To	Start	End
Project Director	José Martín	17/06/2026	22/06/2026

Target		
Name	Platform	Operating System
Vaccine	Hack the Box	Linux

The assignment's objective are

- Obtain initial access to the target system
- Escalate privileges to root
- Recover all available flags
- Document the attack path and identified vulnerabilities

6. Scope

In scope

- Linux operating system
- Web application
- PostgreSQL Database
- Local privilege escalation vectors

7. Assessment summary

Assesment summary	
Findings	Severity
Weak password storage	High
SQL Injection	Critical
Excessive Database Privileges	Critical
Harcoded Database Credentials	High
Credentials reuse	High
Insecure sudo configuration	Critical

8. Detailed Walkthrough

The process had the following phases.

8.1 Reconnaissance

Network reconnaissance identified three exposed services:

- FTP
- SSH
- HTTP

A fingerprinting of the FTP service revealed not only the anonymous authentication was allowed, also the existence of the Backup.zip file. This file contained two compressed files which were index.php and style.css . The index.php file had hardcoded the admin credentials.

The fingerprinting of the HTTP service shown the login panel of a web application

8.2 Exploitation

The admin's credentials were used in the login panel of the web application and allowed to bypass the login form and access to the admin's panel which had a search box that was manually tested if it was vulnerable to SQL Injection. To confirm the searchbox was vulnerable to SQL Injection the sqlmap tool was executed using a HTTP request did to the application web and captured with the Burpsuite tool. The sqlmap tool not only confirmed that the searchbox was vulnerable to multiple SQL Injection, also gave the following information.

- DBMS —> PostgreSQL
- OS —> Linux
- Search parameter —> Vulnerable
- SQL Injection —> Confirmed

Once it was discovered that the web application used PostgreSQL as Database Management System, a fingerprinting of the dbms did not revealed useful information only shown information about cars, however the information gathered about the dbms suggested that it would be possible to use the - - os-shell option of sqlmap to get a operating system command execution.

The - - os-shell option of sqlmap succesfully achieved an operating system command execution. The database user was confirmed to possess DBA privileges, allowing arbitrary commands to be executed. This confirm that the SQL injection vulnerability could be escalated to Remote Code Execution (RCE), significantly increasing the impact of the compromise.

To determine the security context of the obtained operating system shell, the whoami command was executed. The command returned postgres, confirming that arbitrary operating system commands were executed with the privileges of the PostgreSQL service account. Although this did not provide

root-level access, it demonstrated full command execution within the security context of the database service, which could facilitate further post-exploitation activities depending on the host configuration.

8.3 Post – Exploitation

The command execution obtained through sqlmap was sufficient to validate the impact of the SQL injection vulnerability. However, due to the limited interaction capabilities of sqlmap's OS shell, a more stable interactive shell was required to continue host enumeration and privilege escalation.

The first step was check if python3 was available on the target's machine, as the target's machine had installed python3, a python payload was used to generate a stable shell

With the stable shell created, the system of the target's machine was enumerated to gather information about it and find a way to escalate privileges. To do that the `/etc/os-release` file was inspected to identify the Linux distribution and version. This information helps determine default filesystem locations, package management, service configurations and the applicability of privilege escalation techniques specific to the operating system.

The second step was listing the current directory to find possible useful files. The current directory was `/var/lib/postgresql/11/main`. This directory PostgreSQL directory usually stores databases and the main configuration files. The priority was discover new user accounts inside the PostgreSQL, so for the moment this directory was not be listing it. Instead of that the home directory was listed due to usually this directory is the default directory of the user accounts.

The home directory had two subdirectories which were `/simon` and `ftpuser` directories. The `/simon` directory did not have useful information. The `/ftpuser` directory would be listed later. Before do that two techniques to elevated privileges was tested

The first technique was check the SUID bit due to default users execute commands like it was root user, if a program has this bit maybe it could be a way to generate a privilege escalation. Although many files had active de SUID bit any of them had misconfigurations of the SUID bit.

The second technique was check the binary file because if If a binary file can be executed like a root user, it is possible to get a root shell due to that binary file was executed as a root user, so it was checked which binaries could be executed as root user. At the beginning it was not possible to check them because the postgres user credentials were necessary.

Find them was possible because PostgreSQL was the database of a web application so maybe a php configuration file could contains it. The `/var/www/html/dashboard.php` had the credentials of the Postgres user hardcoded. These credentials were considered valuable because password reuse between application services and operating system accounts is a common misconfiguration. Therefore, they were tested against local authentication mechanisms.

With the postgres's credentials it was tested again which binaries could be executed as root user. The postgres could execute command as root user but only with that file. The file was `ALL) /bin/vi /etc/postgresql/11/main/pg_hba.conf`. That was a great opportunity to generate a root shell due to if it was possible to execute vi editor as root user, there is a way to execute scripts inside the vi editor to get a root shell.

Although access was restricted to a specific file, vi provides functionality to execute external commands. Since the editor itself runs with elevated privileges, any spawned shell inherits those privileges, allowing privilege escalation to the root user.

Once the root shell was obtained, it was necessary to check which user was inside the system

- Unordered itemuid=0(root) gid=0(root) groups=0(root)

This resulted in full administrative access to the target system.

8.3.1 Root`s flag

Once the system was fully compromised, the root directory was listed to find the flag file.

- 4 rw——— 1 root root 33 Feb 25 2020 root.txt

The file was readed and the flag was obtained

8.3.2 User`s flag

The only directory that did not be listed was /var/lib/postgresql/11/main, so it was investigated in order to find the flag. That directory had the file user.txt

- /var/lib/postgresql/user.txt

The file was readed and the flag was obtained.

9. Remediation summary

The vulnerabilities found that are recommended to be fixed are

1. Weak password storage

Replace MD5 with a modern password hashing algorithm such as bcrypt or scrypt, using a unique salt for each password

2. SQL Injection

Use parameterized queries (prepared statements). implement server-sides input validation and avoid dynamically building SQL queries with user input

3. Excessive Database Privileges

Apply the principle of least privilege by granting only the permissions required for the application. Remove administrative privileges from application database accounts.

4. Credentials reuse

Enforce the unique password policy for every account and prohibit password reuse across different services

5. Insecure sudo configuration

Restrict sudo permissions following the principle of least privilege, remove unnecessary entries and avoid allowing interactive editors to run as root unless absolutely required.

Immediate remediation of the SQL Injection vulnerability and the insecure sudo configuration should be considered the highest priority.

10. Appendix A: Finding severities

Each finding has been assigned a severity rating of critical, high, medium, low. The rating is based off of an assessment of the priority with which each finding should be viewed and the potential each one has on the confidentiality, integrity and availability of Vaccine's data.

Severity Definitions	
Rating	Severity rating definition
Critical	It allows a full compromise of the system or unauthorized access to highly sensitive information with little or no prior preparation. It can lead to a remote code execution, administrative access or complete control of the target system.
High	Allows significant access to sensitive data or functionality, privilege escalation or lateral movement. Requires an additional condition or has lesser impact than a critical vulnerability.
Medium	It may facilitate subsequent attacks or expose information that is useful to an attacker. The impact on integrity, confidentiality and availability of the data is moderate and typically requires chaining with other vulnerabilities to achieve a full exploitation.
Low	It has a limited impact or is primarily informational. It does not typically grant direct access to the system, but it can assist an attacker during the reconnaissance or enumeration phases.

11. Appendix B : Tools

- Nmap
- curl
- ftp
- gobuster
- zipinfo
- zip2john
- john the ripper
- Burpsuite
- sqlmap
- netcat
- python3
- vi

12. Appendix C : Technical report

See the attached file:

- Vaccine_Technical_Report.pdf